수집 스케줄러 · 백업 변경사항

2026-10-06에 바뀐 내용입니다. 오전 6시에 PC가 꺼져 있어도 수집이 빠지지 않도록 스케줄러를 고쳤고, DB 버전과 팀원 프론트를 개인 저장소에 보관했습니다.

날짜	2026-10-06
브랜치	`jiyoon(cheongyakP)` 커밋 `b1a93bc` (main 미반영)
관련 문서	`docs/261002_6_청약플랫폼_데이터베이스_구조.pdf` · `docs/261002_7_청약플랫폼_덤프파일_가이드.pdf`

목차

1 **한눈에 보기**
오늘 바뀐 것 4가지

2 **수집 스케줄러 수정** 코드
무엇이 문제였고, 이제 어떻게 동작하는지, 실제 확인 결과

3 **새 테이블 collect_runs** DB
수집 실행 기록 — 컬럼과 확인 방법

4 **개인 저장소 백업** 보관
cheongyak-archive의 브랜치·태그·Release 구성, 덤프와 코드 짝 맞추기

5 **현재 상태와 다음 할 일**
최신 행 수, main과의 차이, 남은 작업

1. 한눈에 보기

#	변경	내용
1	**놓친 수집 보충**	오전 6시에 PC가 꺼져 있거나 절전이면, 켜진 뒤 1분 안에 한 번 수집합니다 (이전에는 그날 수집이 그냥 빠짐)
2	**수집 실행 기록**	새 테이블 `collect_runs` — 언제, 왜(정기·보충·수동) 수집했고 어떤 단계가 실패했는지 남깁니다
3	**나흘 치 데이터 보충**	10/03~10/06 오전 6시 수집이 빠져 있던 것을 10/06 11:52~12:02 보충 수집으로 채웠습니다 (실패 0건)
4	**개인 저장소 백업**	`JiYoon241111/cheongyak-archive` (Private)에 DB 버전 코드, 팀원 프론트(main), 덤프 파일 2개를 보관했습니다

4일
자동 수집이 빠졌던 기간 (10/03~10/06)

10분
보충 수집 소요 시간 (11:52~12:02, 실패 0건)

28개
테스트 통과 (스케줄러 테스트 4개 추가)

2. 수집 스케줄러 수정

2-1. 무엇이 문제였나

DB를 개인 저장소에 백업하면서 행 수를 비교하다가, **10월 2일 이후 DB가 한 번도 바뀌지 않은 것**을 발견했습니다. 백엔드 로그에 원인이 남아 있었습니다.

```
2026-10-06 11:05  Run time of job "collect_all (trigger: cron[hour='6', minute='0'] …)" was missed by 5:05:09
```

- **원인**: 오전 6시에 PC가 꺼져 있거나 절전 상태였습니다. APScheduler의 기본 설정은 "정각에 깨어 있을 때만 실행"이라, 늦게 켜지면 그날 수집을 건너뜁니다.
- **영향**: 10/03~10/06 나흘 동안 새 공고·경쟁률·통계가 들어오지 않았습니다. 덤프(10/02)와 DB가 같았던 것도 이 때문입니다.

2-2. 이제 어떻게 동작하나

상황	이전	지금
오전 6시에 PC가 켜져 있음	6시에 수집	같음 — 6시에 수집 (기록: `정기(오전 6시)`)
6시에 **절전**, 나중에 깨어남	그날 건너뜀	깨어나는 즉시 한 번 수집 (기록: `놓친 수집 보충(절전 후, N분 늦음)`)
6시에 **꺼져 있음**, 나중에 켬	그날 건너뜀	백엔드가 켜지고 1분 뒤 한 번 수집 (기록: `놓친 수집 보충(서버 시작 시)`)
며칠 연속 놓침	계속 건너뜀	한 번만 몰아서 수집
이미 수집 중 (이 서버 또는 터미널 수동 실행)	확인 안 함	겹쳐 실행하지 않고 건너뜀

2-3. 어떻게 구현했나 (`backend/app/data_pipeline/scheduler.py`)

경우	방법
절전에서 깨어남	서버가 멈춰 있다가 그대로 이어지므로 스케줄러가 "6시를 놓쳤다"는 것을 압니다 → `misfire_grace_time=None` (아무리 늦어도 실행), `coalesce=True` (여러 번 놓쳐도 한 번만)
PC를 껐다 켬	서버가 새로 시작해 스케줄러는 놓친 것을 모릅니다 → 시작할 때 `collect_runs` 를 보고 **가장 최근 오전 6시 이후 끝난 수집도, 진행 중인 수집도 없으면** 1분 뒤 보충 수집 (`catch_up_needed`)
중복 방지	같은 서버 안에서는 잠금(lock), 다른 곳(터미널 수동 실행 등)은 `collect_runs` 의 `running` 기록으로 확인. 3시간 넘은 `running` 은 중간에 꺼진 것으로 봅니다

테스트 `backend/tests/test_scheduler.py`: 가장 최근 6시 계산 · PC가 꺼져 있었을 때 보충 · 오늘 이미 수집했으면(수동 포함) 보충 안 함 · 수집 중이면 기다리고 3시간 넘으면 보충.

2-4. 실제 확인 결과

코드를 바꾸자 백엔드가 자동으로 다시 시작되며 바로 보충 수집이 돌았습니다.

```
[스케줄러] 10-06 06:00 이후 수집 기록이 없어 11:52에 보충 수집합니다.
[스케줄러] 놓친 수집 보충(서버 시작 시) 시작
  … 청약홈 · 경쟁률 · LH(새 공고·상태 변경 28건만 상세 재수집) · SH · 파인 · 통계 · 공고문/계약금 …
collect_runs: 놓친 수집 보충(서버 시작 시) / done / 11:52 ~ 12:02 / 실패 단계 없음
```

항목	10/02	10/06 보충 후
청약홈 공고 / listings	6,065 / 25,860	6,074 / 25,899
historical_results	346,328	346,411
LH 공고 (상세 원본)	21,327	21,334
SH 게시판	2,621	2,625

notice_costs (계약금)	967	1,010

> **꼭 확인할 것** — 보충 수집도 **백엔드(Docker)가 켜져 있어야** 동작합니다. PC를 켰을 때 Docker Desktop이 자동으로 시작되도록 설정하세요: Docker Desktop → Settings → General → **Start Docker Desktop when you sign in**.

3. 새 테이블 collect_runs

전체 수집(`collect_all`)이 실행될 때마다 1행이 생깁니다. 스케줄러가 보충 여부를 판단하는 근거이고, 사람이 "수집이 잘 돌고 있나"를 확인하는 곳이기도 합니다.

컬럼	의미
`trigger`	실행 이유 — `정기(오전 6시)` / `놓친 수집 보충(서버 시작 시)` / `놓친 수집 보충(절전 후, N분 늦음)` / `수동` (터미널에서 직접 실행)
`status`	`running` (진행 중) / `done` (끝남 — 일부 단계가 실패해도 끝까지 돌았으면 done)
`started_at` · `finished_at`	시작·종료 시각 (시간대 포함)
`failed_steps`	실패한 단계 목록 (예: `["save_lh_listings: HTTPError"]`). 문제없으면 NULL

```
-- 최근 수집 기록 (한국 시간)
SELECT id, trigger, status,
       to_char(started_at  AT TIME ZONE 'Asia/Seoul', 'MM-DD HH24:MI') AS 시작,
       to_char(finished_at AT TIME ZONE 'Asia/Seoul', 'MM-DD HH24:MI') AS 종료,
       failed_steps
FROM collect_runs ORDER BY id DESC LIMIT 10;
```

테이블은 백엔드가 시작될 때 자동으로 만들어집니다 (`create_all`). 덤프로 받은 팀원은 2026-10-06 덤프부터 이 테이블이 들어 있습니다.

4. 개인 저장소 백업

팀 저장소와 별도로, 개인 저장소 `github.com/JiYoon241111/cheongyak-archive` (**Private**)에 지금 시점의 두 버전과 데이터를 보관했습니다. 팀 저장소에는 아무 변화가 없습니다.

개인 저장소에서	내용	기준
브랜치 `db-version` (기본)	DB 수집 · 가공 레이어 · 계약금 · 통계 · 스케줄러	팀 `jiyoon(cheongyakP)` 와 같음 (`b1a93bc`)
브랜치 `main-frontend`	팀원 프론트(홈·공고 목록·대시보드·상세·대출·통근·챗봇), LLM 챗·RAG	팀 `main` `d6572ea` (2026-10-06)
태그 `db-archive-20261006` · `main-archive-20261006`	백업한 시점 고정 표시	`43d4430` · `d6572ea`
Release **`db-dump-20261006`** (최신)	`cheongyak_20261006.dump` 63.3MB — 10/06 보충 수집까지 반영	코드 `b1a93bc` 와 짝
Release `db-archive-20261006`	`cheongyak_20261002.dump` 62.7MB — 이전 기록	코드 `43d4430` 와 짝

> **덤프와 코드는 짝을 맞춰 쓰세요.** 덤프는 같은 Release에 적힌 코드 버전과 테이블 구조가 같습니다. 10/06 덤프에는 `collect_runs` 테이블이 추가됐습니다. `main-frontend` (팀 main) 코드와는 테이블 구조가 달라 덤프를 그대로 쓸 수 없습니다.

덤프 검증

두 덤프 모두 시험용 DB에 실제로 복원해 원본과 행 수가 전부 같은 것을 확인했고, 업로드한 파일 크기도 PC 원본과 바이트 단위로 같습니다.

덤프	크기	만들기	복원	검증
`cheongyak_20261002.dump`	62.7MB	15초	42초	16개 테이블·뷰 행 수 일치
`cheongyak_20261006.dump`	63.3MB	13초	34초	17개 테이블·뷰 행 수 일치 (`collect_runs` 포함)

꺼내 보는 방법

```
git clone https://github.com/JiYoon241111/cheongyak-archive.git
git switch main-frontend        # 팀원 프론트 보기
git switch db-version           # DB 버전으로 돌아오기
# 덤프: Release 페이지에서 .dump 파일 클릭 → 복원은 덤프파일 가이드 3장
```

5. 현재 상태와 다음 할 일

5-1. 최신 행 수 (2026-10-06 12:02 보충 수집 후)

테이블	행	테이블	행
listings	126,133	lh_notice_details	21,334
historical_results	346,411	lh_rental_houses	1,715
bankbook_stats	2,041,305	sh_notices	2,625
subscription_stats	2,786	notice_documents	327
loan_products / loan_rate_options	280 / 682	notice_costs / notice_option_costs	1,010 / 257
recommendable_listings	813 (공고 123건)	collect_runs	1

5-2. 팀 main과의 차이

> 이 브랜치에는 main에 없는 커밋이 **19개**, main에는 이 브랜치에 없는 팀원 커밋이 **5개** 있습니다 (LLM 챗·RAG·계약금 API, 프론트 화면, 통근 경로, 공고 목록 4가지 보기). 두 쪽에서 모두 고친 파일이 15개라 합칠 때 충돌이 예상됩니다 — 수집(`save_listings.py` · `applyhome_client.py` 등), 테이블 구조(`models/listing.py` · `historical_result.py`), 서비스(`eligibility_filter.py` · `routes/listings.py`), 계약금 방식.

5-3. 다음 할 일

#	작업	내용
1	main 내용 가져와 합치기	main을 이 브랜치로 가져와 충돌 15개 정리 (main은 건드리지 않고 브랜치 안에서). 계약금은 두 방식(공고문 표 추출·근거 줄 vs 팀원 계약금 API)을 어떻게 합칠지 팀과 결정
2	프론트 목업	팀원이 main에 만든 화면을 기준으로 시작 (`main-frontend` 브랜치에서 참고)
3	Docker 자동 시작 설정	보충 수집이 동작하려면 PC 로그인 시 Docker Desktop이 켜져야 함

수치는 2026-10-06 `cheongyak` DB 조회 결과입니다. 매일 수집으로 행 수는 조금씩 늘고, recommendable_listings는 접수가 끝난 공고가 빠지면서 줄기도 합니다.